Compiler Design

Course Introduction

Professor Yi-Ping You (游逸平)
Department of Computer Science
http://www.cs.nycu.edu.tw/~ypyou/

Administrative Stuff

- Course information
  - Compiler Design
  - Credit: 3
  - Time: ~~9:00~~ 9:10-12:00 on Thursdays
  - Place: EC015
- Course website
  - https://compiler-s26.github.io/
    - The URL is also provided on my Web page
- GitHub is used
    - A GitHub account is required!
    - Sign up if you don't have a GitHub account

Identification Registration

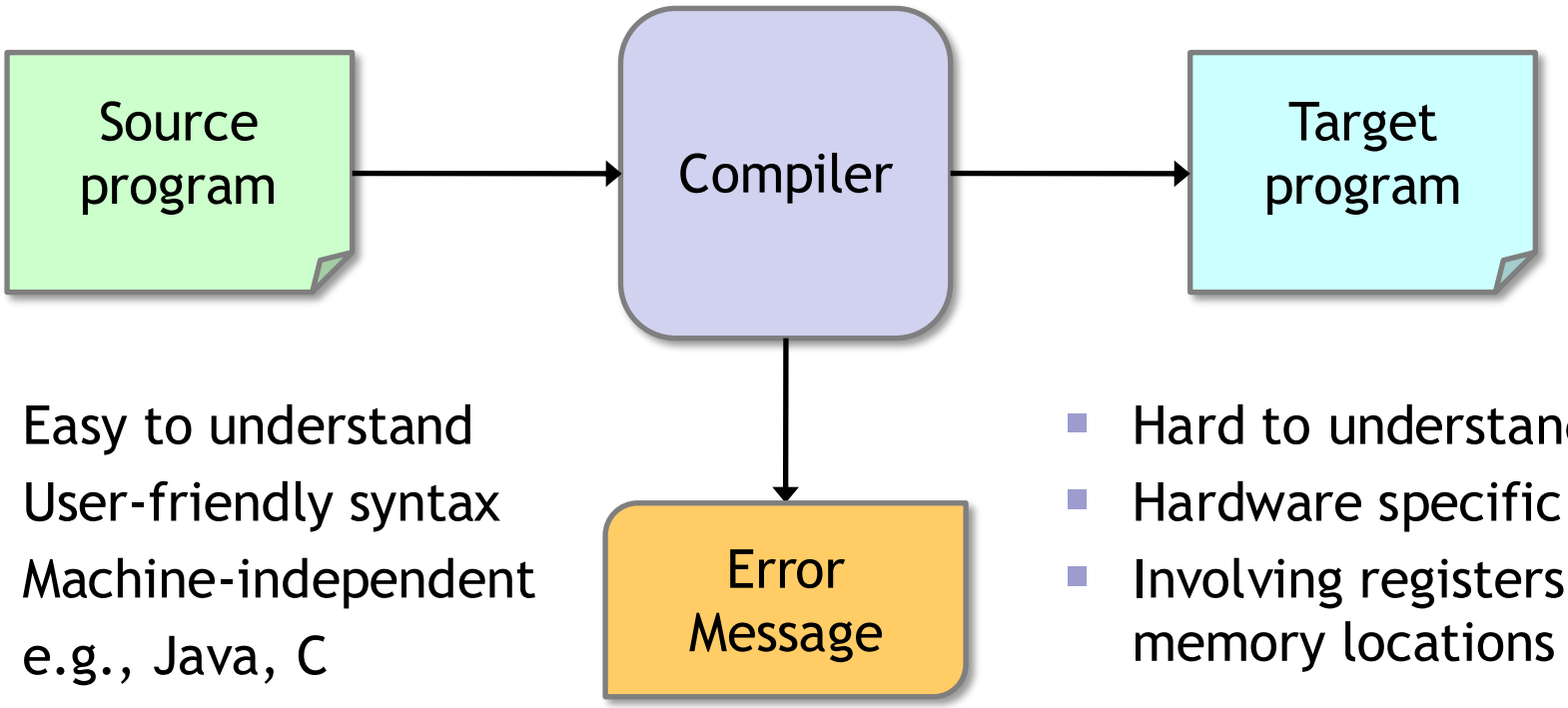
- Google Forms
  - https://forms.gle/eHhHKEFML1331qJ2A

Prerequisites

- Programming Languages, Data Structures, and Assembly Language and System Programming
- You are assumed to have experience with programming, especially in C language

What is a Compiler?

- A language translator



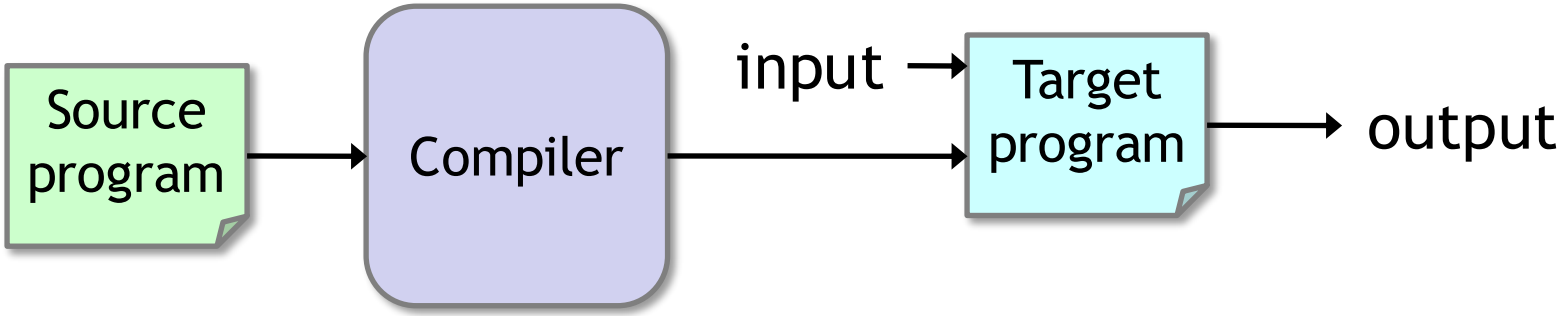
- Easy to understand
- User-friendly syntax
- Machine-independent
- e.g., Java, C

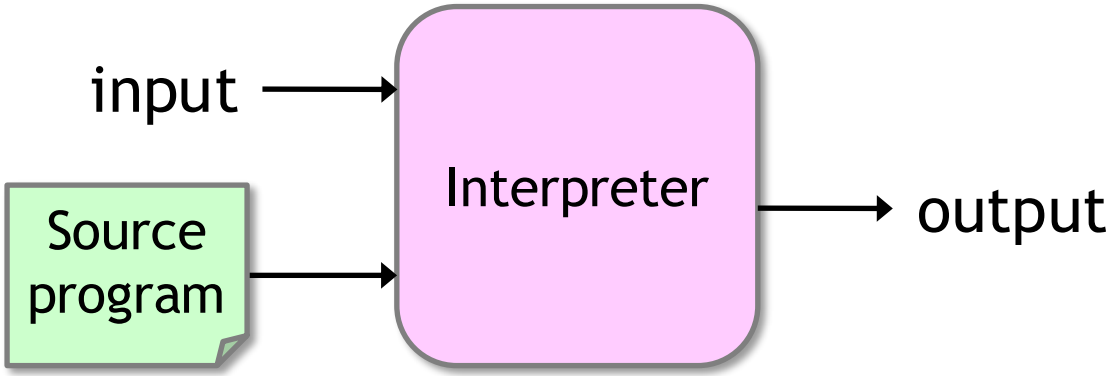
- Hard to understand
- Hardware specific
- Involving registers & memory locations

Terminology

- Compiler:
  - a program that translates an *executable* program in one language into an *executable* program in another language
  - we expect the program produced by the compiler to be better, in some way, than the original
- Interpreter:
  - a program that reads an *executable* program and produces the results of running that program
  - usually, this involves executing the source program in some fashion
- Our course is mainly about compilers but many of the same issues arise in interpreters

Compiler v.s. Interpreter



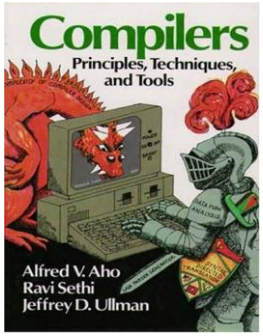


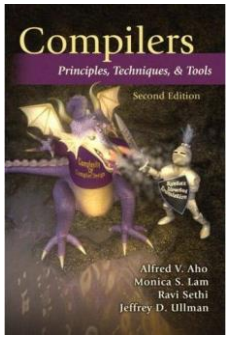
Aims of This Course

- To learn structure of compilers.
- To learn basic techniques used in compiler construction such as lexical analysis, top-down and bottom-up parsing, context-sensitive analysis, code generation, and code optimizations.
- To learn basic data structures used in compiler construction such as abstract syntax trees, symbol tables, three-address code, and stack machines.
- To learn software tools used in compiler construction such as lexical analyzer generators (Lex), and parser generators (Yacc).
- To construct a compiler for a simple language using the above techniques and tools.

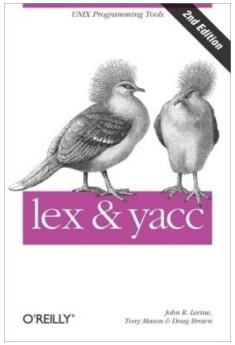
Textbook

- *Compilers: Principles, Techniques, and Tools* (2nd edition), by Aho, Lam, Sethi, and Ullman, Addison Wesley, 2007.
  - A.k.a. dragon book





- Reference books
  - *Lex & Yacc*, by Doug Brown, John Levine, and Tony Mason, O'Reilly Media, 1995.



Lecture Topics (1/2)

- Introduction
  - Course introduction
  - Overview of compilers, phases of a compiler
- Lexical analysis, scanning
  - Regular expressions, lex/flex, start on FSA
  - How lex works, NFAs, DFAs
- Syntax analysis, parsing
  - Context free grammars, ambiguity
  - Yacc/Bison introduction, top-down parsers
  - Top-down parsers, intro to bottom-up parsers
  - Bottom-up parsers
  - LR parsing

Lecture Topics (2/2)

- Context-Sensitive Analysis
  - Syntax-directed definitions, intro to semantic analysis
  - Type checking
- Runtime environments
  - Symbol tables
  - Stack/heap management
- Code generation
  - Generating code for RISC-V assembly language
- Code analysis and optimizations
  - Control-flow analysis and optimizations
  - Data-flow analysis and optimizations

Course Schedule

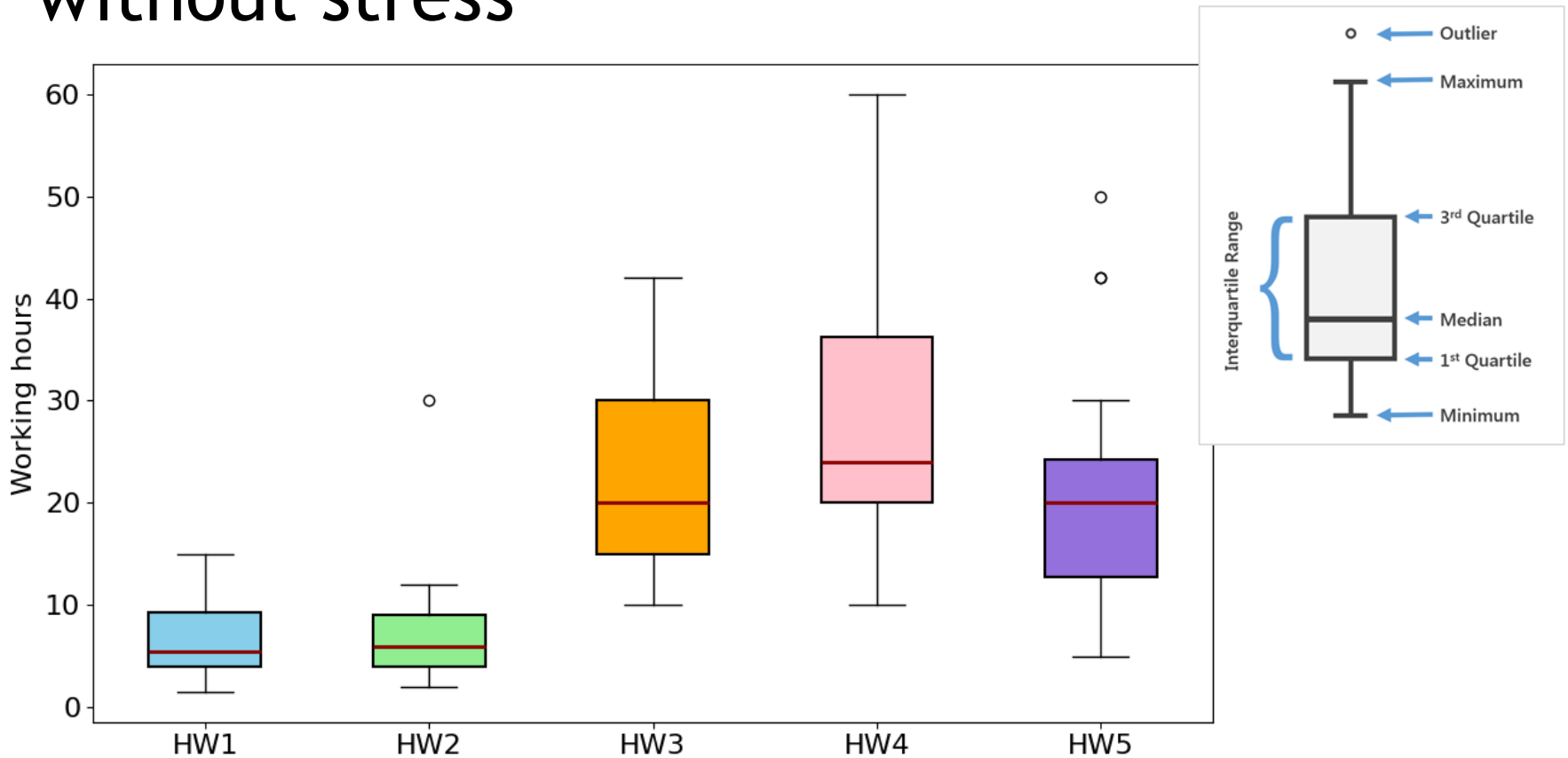
Week	Subject	Readings	Projects
1	Introduction	Ch 1	
2	Regular and Context-Free Languages	Ch 3	
3	Lexical Analysis / Lex - The Lexical-Analyzer Generator	Ch 3	HW1 out, 3/12
4	Top-Down Parsing	Ch 4	HW1 due, 3/19
5	Bottom-Up Parsing	Ch 4	
6	Spring Break	Ch 4	
7	Bottom-Up Parsing / Yacc - The Parser Generator	Ch 5	HW2 out, 4/9
8	Midterm exam		
9	Syntax-Directed Translation	Ch 5	HW2 due, 4/20 HW3 out, 4/23
10	Syntax-Directed Translation / Intermediate-Code Generation	Ch 5/6	
11	Intermediate-Code Generation	Ch 6	
12	Intermediate-Code Generation	Ch 6	HW3 due, 5/11 HW4 out, 5/14
13	Run-Time Environment	Ch 7	
14	Run-Time Environment / Code Generation	Ch 7/8	
15	Code Generation	Ch 8	HW4 due, 6/1 HW5 out, 6/4
16	Final exam		

Grading

- Grades will be assigned based on
  - Midterm exam (25%)
  - Final exam (25%)
  - Course project (50%)
    - In 5 parts (10% each)
      - Bonus points obtained in the parts won't affect the exam scores
    - Slackers beware!
      - The penalty for late homework is **15% per day** (weekends count as 1 day). Late homework will not be accepted after the solutions have been posted.
    - NO PLAGIARISM!
      - Homework assignments must be individual work
- These weights are subject to minor variation

Homework Time Management

- Plan ahead: make sure to allocate your time effectively to complete assignments without stress



Office Hours and TAs

- Contacting me
  - ypyou@cs.nctu.edu.tw
  - Office – EC532
  - Office hours – by appointment
  - Can also catch me right after class
- TAs (Compiler-s26@sslab.cs.nycu.edu.tw)
  - 陳品文 – pwchen@cs.nycu.edu.tw
  - 張博凱 – pkchang@cs.nycu.edu.tw
  - 陳漳昱 – wychen@cs.nycu.edu.tw
  - 黃建樺 – chhuang@cs.nycu.edu.tw
- You should not email directly, even though we give you our email addresses, to the individual members of the staff